

Programação Orientada a Objetos

Kickoff: Requisitos e Arquitetura do GestorOS

[Baixe aqui os exemplos da aula](#)



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Do Exercício ao Sistema

Exercícios de classe resolvem um problema e são descartados. Um sistema permanece: recebe requisitos novos, troca de banco de dados e é mantido por quem não o escreveu. Essa permanência impõe exigências que o exercício dispensa, entre elas organização e capacidade de absorver mudança sem reescrita integral.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

O Domínio: Ordens de Serviço

O projeto do semestre atende uma assistência técnica que recebe equipamentos para conserto. O controle atual, feito em caderno, perde prazos e obriga a recalcular orçamentos à mão. Modelar o domínio significa representar em código as coisas que existem no balcão e as regras que já governam o atendimento.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Classes Candidatas

A modelagem começa pela leitura do enunciado em busca de substantivos com identidade própria e estado que muda ao longo do tempo. Cliente, equipamento, ordem de serviço e técnico atendem ao critério. Telefone e preço não atendem, pois são atributos. Consertar e faturar tampouco, pois são comportamentos.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Coesão: Um Motivo para Mudar

Uma classe coesa concentra atributos e comportamentos que mudam pelo mesmo motivo. Quando uma única classe valida dados, monta consultas e desenha telas, três motivos distintos de mudança convergem sobre o mesmo arquivo, e qualquer alteração arrisca quebrar funcionalidades que nada tinham a ver com ela.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Acoplamento e o Custo da Mudança

Acoplamento mede o quanto uma parte do sistema depende dos detalhes internos de outra. Código fortemente acoplado propaga alterações: trocar o banco de dados obriga a mexer nas telas. Reduzir acoplamento não é elegância acadêmica, mas redução direta do esforço de manutenção ao longo da vida do sistema.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Arquitetura em Camadas

A arquitetura em camadas organiza o sistema em faixas com responsabilidades distintas e dependência em sentido único: apresentação chama serviço, serviço chama persistência, e todas conhecem o modelo. O caminho inverso é proibido. Essa restrição é o que permite substituir uma camada inteira sem tocar nas demais.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

O Que Cada Camada Não Pode Fazer

Definir a camada pelo que ela proíbe é mais eficaz do que defini-la pelo que ela permite. O modelo não conhece banco nem tela. A persistência não decide regra. O serviço não desenha interface. A apresentação não valida negócio. Toda violação dessas quatro proibições aparece como importação indevida.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Pacotes como Expressão da Arquitetura

Em Java a arquitetura deixa de ser diagrama e passa a ser estrutura verificável quando cada camada corresponde a um pacote. A hierarquia de diretórios torna a violação visível, pois uma importação de persistência dentro do modelo salta aos olhos na revisão de código feita por qualquer colega.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Estado Fechado: Por Que Enum

Alguns atributos admitem apenas um conjunto conhecido e finito de valores, como o status de uma ordem de serviço. Representá-los com texto livre transfere a verificação para a disciplina do programador. Representá-los com enumeração transfere a verificação para o compilador, que recusa qualquer valor fora do conjunto declarado.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Camada model: StatusOS

```
// Conceito: o estado da OS e um conjunto FECHADO de valores.  
// Com enum nao existe StatusOS.CONCLUIDAA; com String, existe "Concluidaa".  
package br.edu.fasm.gestoros.model;  
  
public enum StatusOS {  
    ABERTA,  
    EM_ANALISE,  
    EM_EXECUCAO,  
    CONCLUIDA,  
    ENTREGUE,  
    CANCELADA  
}
```



Camada model: Cliente

```
// Conceito: a classe representa uma coisa do mundo real, não uma tela.  
// Atributos privados; o acesso ocorre por métodos públicos.  
public class Cliente {  
  
    private int id;  
    private String nome;  
    private String telefone;  
  
    // O construtor exige o que o objeto precisa para existir.  
    public Cliente(String nome, String telefone) {  
        this.nome = nome;  
        this.telefone = telefone;  
    }  
    // getId, setId, getNome, getTelefone: acesso controlado ao estado  
}
```



A Arquitetura Escrita no Código

```
// Arquivo: src/br/edu/fasm/gestoros/model/package-info.java
// Conceito: a regra da camada fica ao lado do código, não num diagrama solto.

/**
 * Camada de modelo: as entidades do domínio.
 *
 * Guardam estado e não conhecem banco de dados nem tela.
 * Regra: nunca importar nada de dao, view ou service.
 */
package br.edu.fasm.gestoros.model;
```



Ponto de Entrada: GestorOS

```
// Hoje so prova que o ambiente compila e que as entidades ja modelam o problema.  
// Ao fim do semestre esta classe apenas abrira a janela principal.  
public class GestorOS {  
  
    public static void main(String[] args) {  
        Cliente cliente = new Cliente("Marina Alvarenga", "(32) 98811-4402");  
        cliente.setId(1);  
  
        System.out.println("Cliente .....: " + cliente);  
        System.out.println("Status inicial da OS: " + StatusOS.ABERTA);  
    }  
}
```



Atividades

Exercício 1

Cite três classes candidatas do domínio e diga por que telefone não é uma delas.

Exercício 2

Em qual camada mora a decisão de permitir ou não o cancelamento de uma ordem?

Exercício 3

Qual erro o enum evita que a String permite?



Desafio extra

Opcional

Quem quiser ir além do combinado nesta semana:

1. Criar a classe Equipamento no pacote model, com tipo, marca e modelo.
2. Fazer o toString de Cliente incluir o identificador.

Nada disso é cobrado na entrega. O projeto segue completo sem essas adições.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>